PROJECT REPORT

ON

(A Military inventory management systems DBMS)

Submitted by

SHIVAM BHARDWAJ (Reg No.RA2411030030046)

VISHAL KUMAR (Reg No.RA2411030030058)

Under the guidance of

Dr. Anirudha Gaikwad

(Assistant Professor, Department of Computer Science & Engineering)

In partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE & ENGINEERING

of

FACULTY OF ENGINEERING AND TECHNOLOGY

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

(Deemed to be University u/s 3 of UGC Act, 1956)

DELHI-NCR CAMPUS, GHAZIABAD (U.P)

SRM INSTITUTE OF SCIENCE & TECHNOLOGY,

DELHI-NCR CAMPUS

April 2026

(Under Section 3 of UGC Act, 1056)

BONAFIDE CERTIFICATE

This is to certify that the project report entitled "**PROJECT REPORT ON MILITARY INVENTORY MANAGEMENT SYSTEM** " submitted by **Shivam Bhardwaj (Reg No.RA2411030030042) and Vishal Kumar (Reg. No. RA2411030030058)** of **B.Tech — Computer Science and Engineering with specialization in Cyber Security**, in partial fulfilment of the requirements for the award of the degree of **Bachelor of Technology** in **Computer Science and Engineering**, is a record of the original work carried out by them under my supervision and guidance.

This project has been carried out during the academic year **2025–2026** at **SRM Institute of Science and Technology, Delhi-NCR Campus**, and represents the sincere efforts of the students. To the best of my knowledge and belief, this project has not been submitted, in part or full, for the award of any other degree or diploma at this institute or any other educational institution.

The project report embodies the results of investigations conducted by the students and demonstrates their understanding of the subject matter, their ability to carry out systematic research, and their capacity for critical analysis and independent thinking.

I am confident that the knowledge and experience gained by the students through this project will prove valuable for their academic and professional growth in the field of computer science and software development.

I further certify that all necessary ethical guidelines and academic standards have been followed in the preparation of this project report.

SIGNATURE

(Dr. Anirudha Gaikwad)
GUIDE
Assistant Professor, Dept. of CSE
SRM Institute of Science & Technology

SIGNATURE

(Dr. Neelam Singh)
HEAD OF DEPARTMENT (CSE)
Dept. of CSE
SRM Institute of Science & Technology

ACKNOWLEDGEMENT

I take this opportunity to express my profound gratitude and deep regards to all those who have extended their invaluable assistance and support in the successful completion of my project titled **" MILITARY INVENTORY MANAGEMENT SYSTEM"**.

First and foremost, I am deeply indebted to my esteemed guide, **Dr. Anirudha Gaikwad**, Assistant Professor, Department of Computer Science and Engineering, SRM Institute of Science and Technology, Delhi-NCR Campus, for his constant supervision, insightful guidance, and constructive feedback throughout the course of this project. His patience, motivation, and immense knowledge have been pivotal in shaping this work into its final form.

I am equally grateful to **Dr. Neelam Singh**, Head of Department, CSE for her encouragement, academic support, and for providing me with the opportunity and resources to undertake this project successfully.

I extend my sincere thanks to the faculty members and technical staff of the **Department of Computer Science and Engineering**, SRM Institute of Science and Technology, for their cooperation and assistance, which contributed immensely to the smooth execution of my project work.

I would also like to express my heartfelt appreciation to my **parents** and **friends**, whose unwavering support and encouragement kept me motivated throughout this journey. Their belief in my abilities gave me the strength to face challenges and accomplish my goals.

Lastly, I extend my gratitude to all the contributors of online resources and learning platforms such as **GeeksforGeeks**, **W3Schools**, **ChatGPT** and other references that enhanced my understanding and helped me overcome technical difficulties during project development.

This project has been a rewarding learning experience, enabling me to enhance my technical skills, deepen my knowledge of Database Management System and algorithmic thinking, and appreciate the importance of systematic design. I am truly thankful to all who, directly or indirectly, helped make this project a success.

I am truly thankful to all who, directly or indirectly, helped me make this project a success.

Abstract

The Military Inventory Management System (MIMS) is a centralized, multi-tier database application designed to address the complex logistical challenges inherent in defence supply chain operations. Modern military establishments operate across geographically distributed bases at three command levels — Central Command, Regional Depots, and Army Bases — each of which must maintain precise records of weapons, food supplies, and medical equipment.

This system provides a unified PostgreSQL-backed relational database that manages users, units, stock items, batches, transfer requests, approval workflows, automated alerts, and comprehensive audit logging. The schema is designed to enforce strict role- based access control (RBAC) with four roles: Super Admin, Regional Admin, Base Officer, and Auditor. Every data mutation is captured in an immutable audit trail, ensuring accountability and traceability across all command levels.

This report presents the complete database design, entity-relationship model, relational schema, and functional module descriptions derived directly from the Prisma schema definition. The system demonstrates how DBMS principles — normalization, referential integrity, constraint enforcement, and indexing — can be applied to a mission-critical defence environment.

1. Introduction

Inventory management in a military context is significantly more demanding than in civilian logistics. Defence establishments must track assets that range from perishable food supplies with strict expiry dates to weapons requiring scheduled maintenance, all while maintaining a clear chain of custody and ensuring availability during operational readiness assessments.

Traditional paper-based or siloed systems are prone to data redundancy, inconsistency, and delay. A centralized Database Management System resolves these issues by providing a single source of truth accessible to authorized personnel at all command levels. The Military Inventory Management System (MIMS) modelled in this project uses a relational database with well-defined entities, relationships, constraints, and indexes to meet these requirements.

The system is implemented using Prisma ORM with a PostgreSQL backend. The schema defines 14 core tables, 4 enumerations, multiple composite unique constraints, and strategic indexes to optimize query performance. This report analyses every component of that schema and presents it in a format suitable for academic evaluation.

2. Problem Statement

Military logistics operations face the following critical challenges:

- Inventory is distributed across multiple geographically separated bases at three command levels (Central Command, Regional Depots, Army Bases), making consolidated visibility difficult.
- Supply requests from lower-level bases to higher-level depots involve multi-step approval workflows that, when manual, result in delays, loss of documentation, and accountability gaps.
- Items such as food and medicine have expiry dates, while weapons and equipment require periodic maintenance. Without automated tracking, expired or unmaintained assets pose operational and safety risks.
- Transfers of inventory between units — whether fulfilling requests or redistributing surplus — need to be recorded against specific item batches to maintain accurate lot-level traceability.
- Security threats such as unauthorized access, tampering of records, and insider misuse require comprehensive audit logging with user, timestamp, and before/after data capture.
- Role-based access control must prevent Base Officers from approving their own requests, restrict Auditors to read-only access, and reserve system-wide configuration to Super Admins.

The proposed database system addresses all these challenges through a well-normalized relational schema with enforced referential integrity, enumerated status tracking, and a complete audit log model.

3. Objectives

1. Design a normalized relational database schema for centralized military inventory management.
2. Model a hierarchical unit structure (Central Command → Regional Depot → Army Base) using a self-referencing Unit table.
3. Implement a multi-status request and approval workflow (PENDING → APPROVED/REJECTED → DISPATCHED) with full approval history.
4. Track inventory at batch level to support expiry date management and FIFO consumption.
5. Enforce role-based access control with four distinct user roles (SUPER_ADMIN, REGIONAL_ADMIN, BASE_OFFICER, AUDITOR).
6. Provide automated alerting for low stock levels, item expiry, and maintenance due dates.
7. Maintain a tamper-evident audit log recording all CREATE, UPDATE, DELETE, TRANSFER, APPROVE, and REJECT actions.
8. Capture login history including IP address and user agent for security monitoring and forensic analysis.

4. System Overview

The Military Inventory Management System is structured around a three-tier command hierarchy. At the apex is the Central Command, which holds overall inventory authority and can redistribute stock to Regional Depots. Regional Depots serve as intermediate hubs, receiving stock from Central Command and fulfilling requests from Army Bases. Army Bases are the operational endpoints — they consume supplies and raise replenishment requests.

4.1 Command Hierarchy

Level	CommandLevel Enum	Role
Apex Authority	CENTRAL_COMMAND	SUPER_ADMIN
Intermediate Hub	REGIONAL_DEPOT	REGIONAL_ADMIN
Operational Unit	ARMY_BASE	BASE_OFFICER

4.2 Item Categories

The system tracks three categories of items, each with specific lifecycle concerns:

- WEAPONS — Requires scheduled maintenance (maintenanceDueDate tracked per Batch). Alerts of type MAINTENANCE_DUE are generated automatically.
- FOOD — Perishable items with expiry dates (expiryDate tracked per Batch). EXPIRY alerts are generated for approaching expiry.
- MEDICAL — Medical supplies also subject to expiry tracking, with LOW_STOCK alerts when quantity falls below minThreshold.

4.3 Core Workflows

The system supports three primary operational workflows:

- Request Workflow: A Base Officer creates a Request (status = PENDING). A Regional Admin or Super Admin reviews it and transitions status to APPROVED or REJECTED. Upon physical dispatch, status becomes DISPATCHED.
- Transfer Workflow: Inventory can be directly transferred between units without a formal request (e.g., Central Command redistributing surplus to a Regional Depot). Each Transfer generates Transaction records linked to specific Batches.

- Audit Workflow: Every data-modifying action triggers an AuditLog entry capturing the actor (User), affected table, record ID, action type, and old/new JSON snapshots.

5. Database Design Explanation

5.1 Normalization

The schema adheres to Third Normal Form (3NF). Each table contains attributes that depend solely on the primary key, with no transitive dependencies. For example, item metadata (name, category, unit of measure, minimum threshold) is stored in the Item table and referenced by foreign key wherever needed, preventing redundancy across StockItem, RequestItem, Transfer, and Alert tables.

5.2 Primary Keys

All primary keys are CUID strings (generated by default(cuid())), providing collision- resistant, globally unique identifiers suitable for distributed environments. CUIDs are preferable to auto-incremented integers in systems where records may be created at multiple nodes before synchronization.

5.3 Enumerations

Four PostgreSQL enumerations enforce domain integrity at the database level:

Enum	Values
Role	SUPER_ADMIN, REGIONAL_ADMIN, BASE_OFFICER, AUDITOR
CommandLevel	CENTRAL_COMMAND, REGIONAL_DEPOT, ARMY_BASE
ItemCategory	WEAPONS, FOOD, MEDICAL
RequestStatus	PENDING, APPROVED, REJECTED, DISPATCHED
AlertType	LOW_STOCK, EXPIRY, MAINTENANCE_DUE
ActionType	CREATE, UPDATE, DELETE, TRANSFER, APPROVE, REJECT

5.4 Referential Integrity

All foreign keys are enforced at the database level through Prisma relation directives, which translate to SQL FOREIGN KEY constraints. The RequestItem table uses onDelete: Cascade —

when a Request is deleted, its associated RequestItems are automatically removed, preventing orphaned records.

5.5 Unique Constraints and Indexes

- User.email — ensures no duplicate accounts.
- Item.code — ensures each item has a unique catalogue code.
- Unit.name and Unit.code — prevent duplicate organizational units.
- Batch.batchCode — ensures lot-level traceability.
- StockItem(baseId, itemId) — composite unique constraint ensuring one stock record per item per base.
- Indexes on (baseId, itemId), (status, fromUnitId), (unitId, isRead), (tableName, recordId), (userId, createdAt) optimize frequent query patterns.

6. Entity Analysis

The schema defines 14 core entities. Each is analysed below with its attributes, keys, and constraints.

6.1 User

Represents all system users across command levels. Supports brute-force protection via failedAttempts and lockedUntil.

Attribute	Type	Constraint	Description
id	String (CUID)	PK	Unique user identifier
name	String	NOT NULL	Full name of the user
email	String	UNIQUE, NOT NULL	Login email address
passwordHash	String	NOT NULL	Bcrypt hashed password
role	Role (Enum)	NOT NULL	RBAC role assignment
commandLevel	CommandLevel (Enum)	NOT NULL	Operational command tier
unitId	String	FK → Unit, NULL	Assigned organizational unit
failedAttempts	Int	DEFAULT 0	Brute-force counter
lockedUntil	DateTime	NULLABLE	Account lockout expiry
isActive	Boolean	DEFAULT true	Soft-disable flag

6.2 Unit

Represents organizational units in a self-referencing hierarchy. A Unit can have one parent and many children.

Attribute	Type	Constraint	Description
id	String (CUID)	PK	Unique unit identifier
name	String	UNIQUE, NOT NULL	Official unit name
code	String	UNIQUE, NULLABLE	Short unit code

level	CommandLevel (Enum)	NOT NULL	Tier in the hierarchy
parentId	String	FK → Unit (self), NULL	Parent unit reference

6.3 Item

Defines the master catalogue of inventory items. Supports soft deletion via isDeleted flag.

Attribute	Type	Constraint	Description
id	String (CUID)	PK	Unique item identifier
name	String	NOT NULL	Item name
code	String	UNIQUE, NOT NULL	Unique catalogue code
category	ItemCategory (Enum)	NOT NULL	WEAPONS / FOOD / MEDICAL
unit	String	NOT NULL	Unit of measure (kg, pcs, etc.)
minThreshold	Int	NOT NULL	Low-stock alert trigger level
specifications	Json	NULLABLE	Technical specifications
isDeleted	Boolean	DEFAULT false	Soft delete flag

6.4 StockItem

Represents the current inventory level of an Item at a specific Unit (base). The composite unique constraint (baseId, itemId) ensures exactly one stock record per item per base.

Attribute	Type	Constraint	Description
id	String (CUID)	PK	Surrogate key
baseId	String	FK → Unit, NOT NULL	Owning unit
itemId	String	FK → Item, NOT NULL	Referenced item
quantity	Int	DEFAULT 0	Available quantity
reservedQty	Int	DEFAULT 0	Quantity reserved for pending requests
(baseId, itemId)	Composite	UNIQUE	One record per item per base

6.5 Batch

Represents a physical lot/batch of an Item received into inventory. Tracks original and remaining quantities, manufacture date, expiry date, and maintenance due date.

Attribute	Type	Constraint	Description
id	String (CUID)	PK	Batch identifier
itemId	String	FK → Item, NOT NULL	Item this batch belongs to
batchCode	String	UNIQUE, NOT NULL	Lot identification code
quantity	Int	NOT NULL	Original quantity in batch
remainingQty	Int	NOT NULL	Current remaining quantity
expiryDate	DateTime	NULLABLE	Expiry (FOOD/MEDICAL)
maintenanceDueDate	DateTime	NULLABLE	Next maintenance (WEAPONS)
isExhausted	Boolean	DEFAULT false	Batch fully consumed flag

6.6 Request

Represents a formal supply request from one unit to another. Tracks emergency flag for priority processing.

Attribute	Type	Constraint	Description
id	String (CUID)	PK	Request identifier
fromUnitId	String	FK → Unit, NOT NULL	Requesting unit
toUnitId	String	FK → Unit, NOT NULL	Supplying unit
createdById	String	FK → User, NOT NULL	Officer who raised the request
status	RequestStatus (Enum)	DEFAULT PENDING	Workflow state
isEmergency	Boolean	DEFAULT false	Emergency priority flag

6.7 RequestItem

A line-item within a Request. Records the requested quantity and the approved quantity (which may differ). Uses CASCADE delete tied to the parent Request.

6.8 ApprovalLog

Records every approval/rejection action taken on a Request, including the approver's identity, action string, and optional remarks. Provides a full decision audit trail per request.

6.9 Transfer

Represents a direct inventory transfer between two units without a formal request. Can be used by SUPER_ADMIN or REGIONAL_ADMIN for redistribution. Each Transfer is linked to one Item and one initiating User.

6.10 Transaction

A low-level ledger entry linked to both a Batch and a Transfer. Records the type of movement (e.g., ISSUE, RECEIPT), quantity, and the base where it occurred. Enables full traceability from stock balance back to the originating batch and transfer.

6.11 AuditLog

Immutable log of all system actions. Stores old and new data as JSON snapshots, enabling rollback analysis and forensic audit. Indexed on (tableName, recordId) and (userId, createdAt) for efficient filtering.

6.12 Alert

System-generated notifications for LOW_STOCK, EXPIRY, and MAINTENANCE_DUE conditions. Linked to both a Unit and optionally an Item. The isRead flag tracks acknowledgement status. Indexed on (unitId, isRead) for fast unread alert queries.

6.13 LoginHistory

Records every login attempt (successful or failed) with IP address and user agent string. Supports security monitoring, detection of compromised accounts, and forensic investigation.

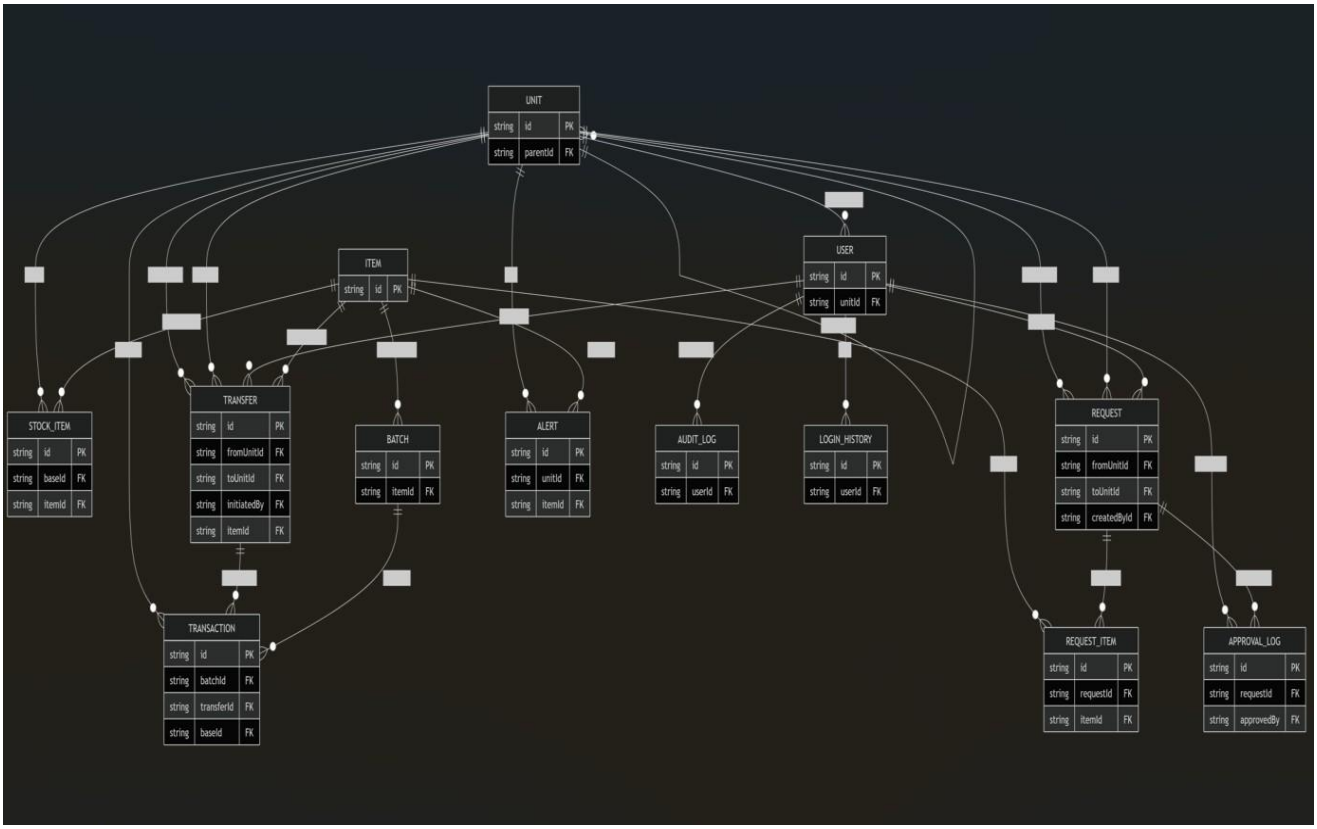
7. ER Diagram

The Entity-Relationship diagram below is presented in Mermaid.js erDiagram notation, which follows Crow's Foot cardinality symbols. The diagram can be rendered using the Mermaid Live Editor at mermaid.live or embedded in any Mermaid-compatible renderer.

7.1 Cardinality Legend

Notation	Meaning	Example
‖--‖	Exactly one to exactly one	N/A in this schema
‖--o{	One (mandatory) to zero or many	Unit to StockItem
}o--o{	Zero or many to zero or many	N/A (no M:N without junction)

7.2 Mermaid ER Diagram Code



7.3 Relationship Summary Table

Parent Entity	Child Entity	Type	FK in Child
Unit	Unit (self)	1:N (self)	parentId → Unit.id
Unit	User	1:N	unitId → Unit.id
Unit	StockItem	1:N	baseId → Unit.id
Unit	Request (from)	1:N	fromUnitId → Unit.id
Unit	Request (to)	1:N	toUnitId → Unit.id
Unit	Transfer (from/to)	1:N	fromUnitId / toUnitId
Unit	Alert	1:N	unitId → Unit.id
Unit	Transaction	1:N	baseId → Unit.id
User	LoginHistory	1:N	userId → User.id
User	Request	1:N	createdById → User.id
User	ApprovalLog	1:N	approvedBy → User.id
User	AuditLog	1:N	userId → User.id
User	Transfer	1:N	initiatedBy → User.id
Item	StockItem	1:N	itemId → Item.id
Item	Batch	1:N	itemId → Item.id
Item	RequestItem	1:N	itemId → Item.id
Item	Alert	1:N (optional)	itemId → Item.id (NULL)
Item	Transfer	1:N	itemId → Item.id
Request	RequestItem	1:N (CASCADE)	requestId → Request.id
Request	ApprovalLog	1:N	requestId → Request.id
Batch	Transaction	1:N	batchId → Batch.id (NULL)
Transfer	Transaction	1:N	transferId → Transfer.id (NULL)

8. Relational Schema

The following relational schema notation is used: PK = Primary Key, FK = Foreign Key, U = Unique, NN = Not Null, N = Nullable.

users

```
users(
  id           CUID          PK,
  name         VARCHAR       NN,
  email        VARCHAR       NN, U,
  password_hash VARCHAR      NN,
  role         ENUM(Role)    NN,
  command_level ENUM(CommandLevel) NN,
  unit_id      CUID          FK → units(id), N,
  avatar_url  VARCHAR        N,
  failed_attempts INT        NN  DEFAULT 0,
  locked_until  TIMESTAMP    N,
  last_login_at TIMESTAMP    N,
  is_active    BOOLEAN       NN  DEFAULT true,
  created_at   TIMESTAMP     NN  DEFAULT NOW(),
  updated_at   TIMESTAMP     NN  AUTO-UPDATE
)
```

units

```
units(
  id          CUID           PK,
  name        VARCHAR        NN, U,
  code        VARCHAR        N,  U,
  level       ENUM(CommandLevel) NN,
  parent_id   CUID           FK → units(id), N,
  created_at TIMESTAMP       NN  DEFAULT NOW()
)
```

items

```
items(
  id               CUID           PK,
  name             VARCHAR        NN,
  code             VARCHAR        NN, U,
  category         ENUM(ItemCategory) NN,
  unit             VARCHAR        NN,
  min_threshold    INT            NN,
```

```
  specifications JSONB          N,
  is_deleted      BOOLEAN       NN  DEFAULT false,
  deleted_at      TIMESTAMP     N,
  created_at      TIMESTAMP     NN  DEFAULT NOW(),
  updated_at      TIMESTAMP     NN  AUTO-UPDATE
)
```

stock_items

```
stock_items(
  id              CUID      PK,
  base_id         CUID      NN   FK → units(id),
  item_id         CUID      NN   FK → items(id),
  quantity        INT       NN   DEFAULT 0,
  reserved_qty    INT       NN   DEFAULT 0,
  last_updated TIMESTAMP NN DEFAULT NOW() AUTO-UPDATE,
  UNIQUE (base_id, item_id)
)
```

batches

```
batches(
  id                   CUID      PK,
  item_id              CUID      NN  FK → items(id),
  batch_code           VARCHAR   NN, U,
  quantity             INT       NN,
  remaining_qty        INT       NN,
  manufacture_date     TIMESTAMP N,
  expiry_date          TIMESTAMP N,
  maintenance_due_date TIMESTAMP N,
  received_at          TIMESTAMP NN  DEFAULT NOW(),
  is_exhausted         BOOLEAN   NN  DEFAULT false
)
```

requests

```
requests(
  id            CUID              PK,
  from_unit_id  CUID              NN  FK → units(id),
  to_unit_id    CUID              NN  FK → units(id),
  created_by_id CUID              NN  FK → users(id),
  status        ENUM(RequestStatus) NN DEFAULT 'PENDING',
  is_emergency  BOOLEAN           NN  DEFAULT false,
  remarks       TEXT              N,
  created_at    TIMESTAMP         NN  DEFAULT NOW(),
  updated_at    TIMESTAMP         NN  AUTO-UPDATE
)
```

request_items

```
request_items(
  id          CUID     PK,
  request_id  CUID     NN  FK → requests(id)  ON DELETE CASCADE,
  item_id     CUID     NN  FK → items(id),
  quantity    INT      NN,
  approved    INT      N
)
```

approval_logs

```
approval_logs(
  id          CUID      PK,
  request_id  CUID      NN  FK → requests(id),
  approved_by CUID      NN  FK → users(id),
  action      VARCHAR   NN,
  remarks     TEXT      N,
  created_at  TIMESTAMP NN  DEFAULT NOW()
)
```

transfers

```
transfers(
  id            CUID      PK,
  from_unit_id  CUID      NN  FK →  units(id),
  to_unit_id    CUID      NN  FK →  units(id),
  initiated_by  CUID      NN  FK →  users(id),
  item_id       CUID      NN  FK →  items(id),
  quantity      INT       NN,
  remarks       TEXT      N,
  created_at    TIMESTAMP NN  DEFAULT NOW()
)
```

transactions

```
transactions(
  id          CUID      PK,
  batch_id    CUID      N   FK → batches(id),
  transfer_id CUID      N   FK → transfers(id),
  type        VARCHAR   NN,
  quantity    INT       NN,
  base_id     CUID      NN  FK → units(id),
  created_at  TIMESTAMP NN  DEFAULT NOW()
)
```

audit_logs

```
audit_logs(
  id            CUID              PK,
  user_id       CUID              NN  FK → users(id),
  table_name    VARCHAR           NN,
  record_id     VARCHAR           NN,
  action        ENUM(ActionType) NN,
  old_data      JSONB             N,
  new_data      JSONB             N,
  ip_address    VARCHAR           N,
  created at    TIMESTAMP         NN  DEFAULT NOW()
)
```

alerts

```
alerts(
  id            CUID            PK,
  unit_id       CUID            NN  FK → units(id),
  type          ENUM(AlertType) NN,
  message       TEXT            NN,
  item_id       CUID            N   FK → items(id),
  is_read       BOOLEAN         NN  DEFAULT false,
  created_at    TIMESTAMP       NN  DEFAULT NOW()
)
```

login_histories

```
login_histories(
  id            CUID        PK,
  user_id       CUID        NN  FK →  users(id),
  ip_address    VARCHAR     N,
  user_agent    TEXT        N,
  success       BOOLEAN     NN,
  created_at    TIMESTAMP NN  DEFAULT NOW()
)
```

9. Functional Modules

9.1 User Management & Authentication

Entities involved: User, LoginHistory

- User registration with role and command level assignment by SUPER_ADMIN.
- Password stored as bcrypt hash (passwordHash). Plain-text passwords are never stored.
- Failed login attempts tracked via failedAttempts counter; account locked via lockedUntil timestamp after threshold breach.
- Every login attempt (success or failure) recorded in LoginHistory with IP address and user agent.
- isActive flag enables soft-disabling accounts without deletion, preserving referential integrity.

9.2 Organizational Unit Management

Entities involved: Unit

- Self-referencing hierarchy: CENTRAL_COMMAND has no parent; REGIONAL_DEPOT references CENTRAL_COMMAND; ARMY_BASE references REGIONAL_DEPOT.
- A unit's parentId is NULL for Central Command, establishing the root of the hierarchy.
- Unit codes provide short identifiers for display in reports and alerts.

9.3 Inventory Management

Entities involved: Item, StockItem, Batch

- Item master catalogue defines item metadata including minimum threshold (minThreshold) for low-stock alert triggering.
- StockItem maintains a running balance (quantity) and a reserved quantity (reservedQty) for approved-but-not-dispatched requests, preventing over-allocation.
- Batch model supports FIFO inventory consumption — batches with earlier expiry dates are consumed first. remainingQty is decremented as Transactions are posted.
- Soft delete on Item (isDeleted) prevents catalogue records from being hard- deleted while retaining historical references.

9.4 Request & Approval Workflow

Entities involved: Request, RequestItem, ApprovalLog

- A BASE_OFFICER creates a Request referencing fromUnitId (own base) and toUnitId (supplying depot). isEmergency flag can elevate priority.
- Each request line is stored as a RequestItem with requested quantity and optional approved quantity.
- REGIONAL_ADMIN or SUPER_ADMIN reviews the request and logs each decision in ApprovalLog. The action field is a free-text string (e.g., 'APPROVED — partial, reduced to 200 units').
- Status transitions: PENDING → APPROVED → DISPATCHED, or PENDING → REJECTED. The ApprovalLog records every status change with actor and timestamp.
- Cascade delete on RequestItem ensures consistency — deleting a cancelled Request removes all associated line items.

9.5 Transfer Module

Entities involved: Transfer, Transaction

- Transfers model direct inter-unit stock movements without formal request, e.g., rebalancing surplus from REGIONAL_DEPOT to ARMY_BASE.
- Each Transfer records fromUnitId, toUnitId, itemId, quantity, and the initiating user.
- Transaction records link each transfer to the specific Batch from which items are drawn, supporting lot-level traceability.
- Transaction.type field distinguishes ISSUE (outgoing) from RECEIPT (incoming) movements at each base.

9.6 Alert & Notification Module

Entities involved: Alert

- LOW_STOCK: Triggered when StockItem.quantity falls below Item.minThreshold. Alert linked to unitId and itemId.
- EXPIRY: Triggered when Batch.expiryDate is within a configurable threshold (e.g., 30 days). Applicable to FOOD and MEDICAL categories.
- MAINTENANCE_DUE: Triggered when Batch.maintenanceDueDate is approaching. Applicable to WEAPONS category.
- isRead flag tracks acknowledgement. The index on (unitId, isRead) enables efficient retrieval of unread alerts per unit.

9.7 Audit Log Module

Entities involved: AuditLog

- Every CREATE, UPDATE, DELETE, TRANSFER, APPROVE, and REJECT action across any table generates an AuditLog entry.
- oldData and newData stored as JSONB snapshots provide a before/after record for forensic reconstruction.
- IP address capture enables correlation with LoginHistory for detection of compromised sessions.
- Read-only AUDITOR role users can query AuditLog without modifying any data.

9.8 Reporting Module

Derived from: All tables

- Stock Status Report: JOIN StockItem + Item + Unit to show current inventory levels across all bases, filtered by category or base.
- Request History Report: JOIN Request + RequestItem + ApprovalLog to trace the full lifecycle of supply requests.
- Transfer Ledger Report: JOIN Transfer + Transaction + Batch to reconcile stock movements against batch lots.
- Expiry Calendar: Query Batch WHERE expiryDate < NOW() + INTERVAL '30 days' AND isExhausted = false.
- Audit Trail Report: Query AuditLog by userId, tableName, date range for compliance and investigation.

10. Advantages of the System

Advantage	Technical Basis
Centralized Visibility	Single PostgreSQL database with hierarchical Unit model provides real-time inventory visibility across all command levels.
Data Integrity	Foreign key constraints, UNIQUE constraints, and enumerations prevent invalid or inconsistent data at the database level, not just the application layer.
Batch Traceability	Batch + Transaction model enables FIFO consumption tracking, lot recall capability, and expiry/maintenance management.
Accountability	AuditLog with JSON snapshots and ApprovalLog create an immutable chain of custody for all inventory movements and decisions.
Security	RBAC via Role enum, brute-force lockout via failedAttempts/lockedUntil, and LoginHistory with IP/UA capture provide layered security.
Scalability	CUID primary keys and strategic composite indexes support high-volume concurrent operations without hotspot contention.
Normalized Design	3NF normalization eliminates redundancy. Item metadata stored once and referenced by FK prevents update anomalies.
Soft Deletion	isDeleted on Item and isActive on User preserve historical data and referential integrity while logically removing records.
Emergency Handling	isEmergency flag on Request enables priority queuing in approval workflows without schema changes.
Conflict Prevention	reservedQty in StockItem prevents double-allocation by tracking quantities committed to pending requests.

11. Conclusion

The Military Inventory Management System demonstrates a comprehensive application of relational database design principles to a real-world, mission-critical domain. The schema successfully models the complexity of a three-tier military command structure — from Central Command down to individual Army Bases — within a single normalized database.

The design achieves all stated objectives: hierarchical unit management through a self- referencing Unit table, multi-step request and approval workflows with complete ApprovalLog history, lot-level inventory tracking via Batch and Transaction entities, automated alerting for LOW_STOCK, EXPIRY, and MAINTENANCE_DUE conditions, and a tamper-evident audit trail with JSON snapshots for forensic compliance.

From a DBMS perspective, the schema demonstrates correct application of Third Normal Form, appropriate use of enumerated types for domain enforcement, composite unique constraints to prevent semantic duplicates, strategic B-tree indexing for query performance optimization, and cascade delete semantics for maintaining referential integrity.

The brute-force protection mechanism (failedAttempts, lockedUntil), IP-correlated login history, and role-based access control reflect security-aware database design — particularly critical in defence environments where unauthorized access can have serious operational consequences.

In summary, the MIMS schema is a production-ready database design that balances normalization, performance, security, and operational completeness. It provides a solid foundation for building a full-stack military logistics application and serves as a rigorous example of applied database management system design.

References

9. Ramez Elmasri and Shamkant B. Navathe, "Fundamentals of Database Systems", 7th Edition, Pearson Education, 2015.
10. Abraham Silberschatz, Henry F. Korth, S. Sudarshan, "Database System Concepts", 7th Edition, McGraw-Hill, 2020.
11. Prisma ORM Documentation — Schema Reference, Prisma Inc., 2024. Available: https://www.prisma.io/docs/orm/prisma-schema
12. PostgreSQL 16 Documentation — Constraints and Indexes, The PostgreSQL Global Development Group, 2024. Available: https://www.postgresql.org/docs/
13. Mermaid.js Documentation — Entity Relationship Diagrams, 2024. Available: https://mermaid.js.org/syntax/entityRelationshipDiagram.html